Coursework
SystemVerilog Design of an Application Specific Embedded Processor

Introduction

This exercise is done individually and the assessment is:

- By formal report describing the final design, its development, implementation and testing.
- By a laboratory demonstration of the final design on an Altera FPGA development system

Use the following code for the top-level module picoMIPS4test.sv and the clock divider counter.sv. The purpose of the clock divider is to eliminate bouncing effects of the mechanical switches which are used to input data as outlined by the pseudocode below.

File picoMIPS4test.sv:

```
// synthesise to run on Altera DE1 for testing and demo
module picoMIPS4test(
  input logic fastclk,  // 50MHz Altera DE0 clock
  input logic [9:0] SW, // Switches SW0..SW9
  output logic [7:0] LED); // LEDs

  logic clk; // slow clock, about 10Hz

  counter c (.fastclk(fastclk),.clk(clk)); // slow clk from counter

  // to obtain the cost figure, synthesise your design without the counter
  // and the picoMIPS4test module using Cyclone V 5CSEMA5F31C6 as target
  // and make a note of the synthesis summary
  picoMIPS myDesign (.clk(clk), .SW(SW),.LED(LED));

endmodule
```

File counter.sv:

```
// counter for slow clock
module counter #(parameter n = 24) //clock divides by 2^n, adjust n if
necessary
  (input logic fastclk, output logic clk);

logic [n-1:0] count;

always_ff @(posedge fastclk)
    count <= count + 1;

assign clk = count[n-1]; // slow clock

endmodule
```

The objective of this exercise is to design an 8-bit implementation of an application specific picoMIPS which implements the 1-dimensional gaussian smoothing convolution algorithm described below using the smallest possible hardware. A NISC implementation is also allowed.

You may modify the picoMIPS architecture extensively if it helps to reduce the cost figure (defined below). However, when modifying the architecture, bear in mind that a processor is still required to implement the required gaussian convolution algorithm, not a dedicated hardware design. A processor is characterised by the presence of two separate and discernible parts: the control path and the data processing path. The control path should contain a program memory which stores the machine program of the algorithm.

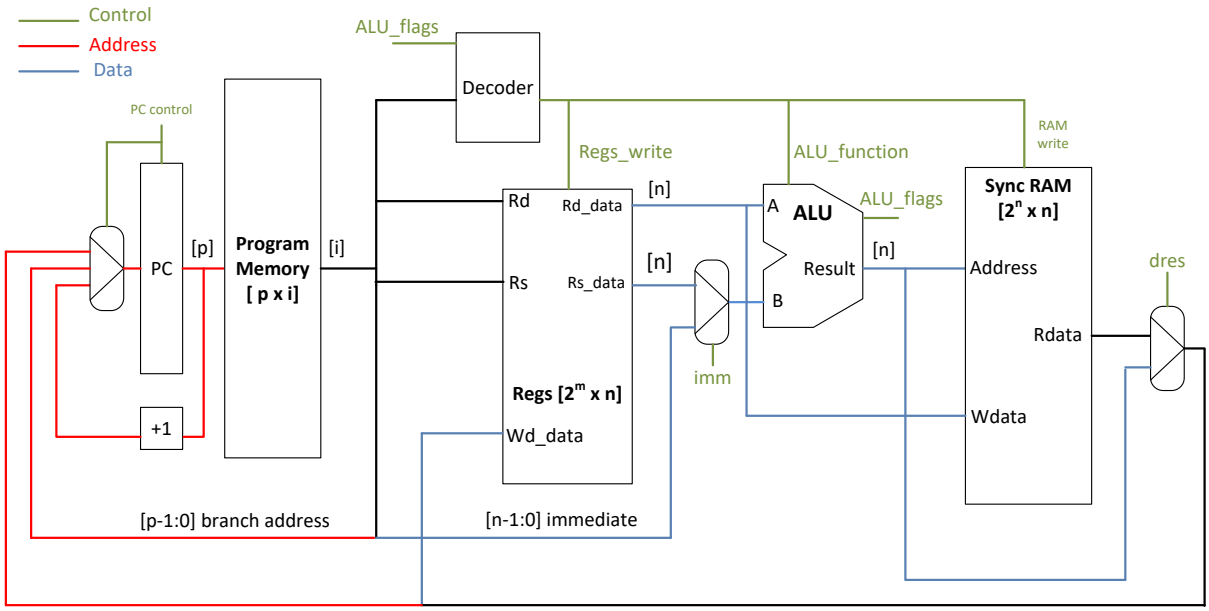

Figure 1. picoMIPS general architecture..

The design should be as small as possible in terms of FPGA resources but sufficient to implement the application described below. The size cost function of the design is defined as follows:

*Cost = number of ALMs (Adaptive Logic Modules) used + 500*max(0,number Variable Precision DSP Blocks in 9x9 multiplier mode used – 2) +30 x Kbits of RAM used*

Each ALM has 2 flip-flops hence flip-flops are included in the above cost figure. The cost figure should be calculated for Altera Cyclone V SoC 5CSEMA5F31C6 device (i.e. the Altera FPGA on the DE1 Development Kit) and should be as low as possible. If one DSP block in 9x9 multiplier mode is used in your implementation, up to three 9x9 multipliers that the block can provide are 'free', i.e. they do not contribute to the size cost. . You **may not** use any other hardware in the DSP blocks in your design, just the multipliers. To demonstrate the cost figure of your design show in your report Altera Quartus synthesis statistics for Cyclone V SoC 5CSEMA5F31C6.

To facilitate lab demonstrations and the cost figure calculation, structure your design as shown in Figure 2 below. Calculate the cost figure only for the picoMIPS module which you develop.

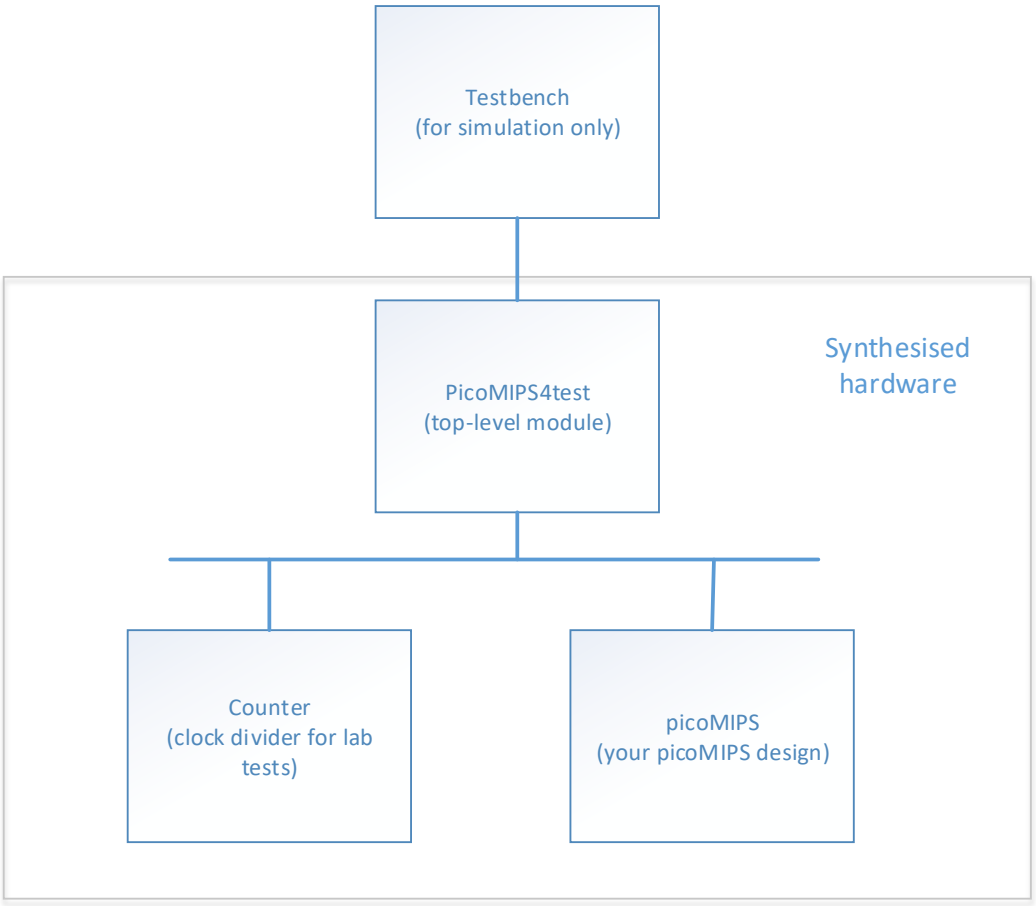

Figure 2. Synthesised design structure.

ELEC6234 lecture slides will describe the picoMIPS architecture in detail and provide SystemVerilog coding suggestions for the picoMIPS blocks. An additional functionality to input 8-bit data and to output 8-bit results will be required as described below. You may design your own instruction set and modify the instruction format in any way you wish. You may also modify the architecture if it helps to reduce the cost figure. However, when modifying the architecture, bear in mind that a processor is still required to implement the affine transformation algorithm outlined below, not a dedicated hardware design. A processor is characterised by the presence of two separate and discernible parts: the control path and the data processing path. The control path should contain a program memory which stores the machine program of the algorithm.

One dimensional Gaussian smoothing

There are many methods of smoothing noisy waveforms but here we want to implement a process of averaging data points in a noisy waveform with their neighbouring data points using convolution. The desired effect is blurring rapid transitions, i.e. reducing the noise in the data. Smoothing can be compared to low pass filtering because it removes high frequency components from the noisy waveform's spectrum. The convolution in Gaussian smoothing, a.k.a as Gaussian blur, is a mathematical operation on the noisy waveform and a Gaussian kernel as explained below. We start with the Gaussian distribution (or normal distribution) function which will define the convolution kernel to be applied to each sample of the waveform. In one dimension the Gaussian function has the following form.

$$G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$$

For $\sigma = 1$ the Gaussian distribution function has the following form:

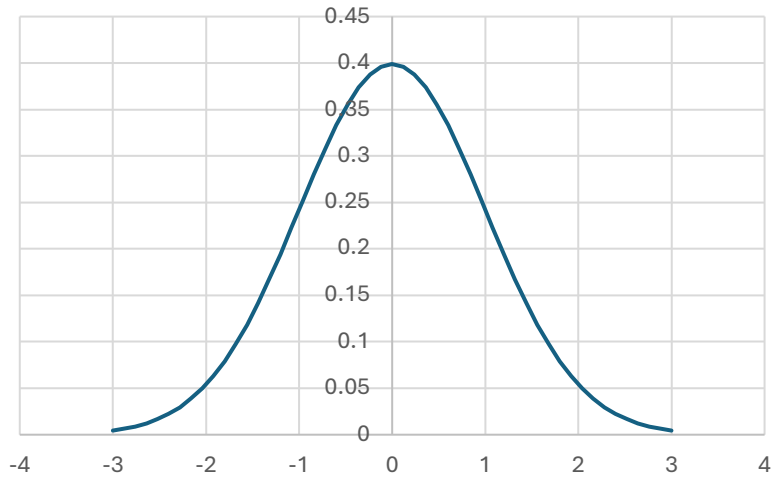

Figure 1. Gaussian distribution function.

The kernel is usually obtained from the Gaussian distribution by selecting a small number of discrete points and sometimes scaling them to amplify or attenuate the convolution result as desired. In this coursework we will use the following 5-point Gaussian kernel: [0.1358, 0.2284, 0.2717, 0.2284, 0.1358]. To avoid fractions, we will scale the kernel by 7 bits, i.e. each value is multipled by 128. So the kernel we will use in our calculations is:

K=[17,29,35,28,17]

In 8-bit 2's complement binary notation these five kernel values are: 00010001, 00011101, 00100011, 00011101, 00010001 or in hexadecimal: 11,1D,23,1D,11.

The corresponding graph of the kernel before scaling is:

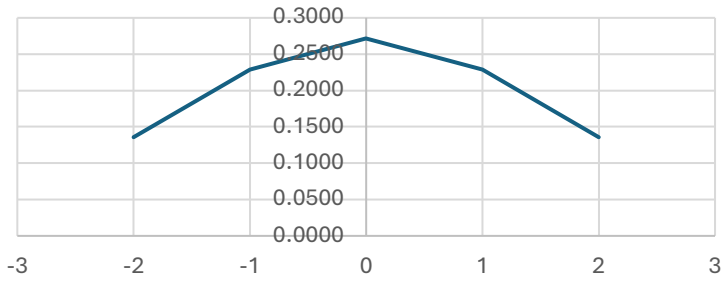

Figure 2. 5-point Gaussian kernel.

The noisy waveform to be smoothed is a sine wave consisting of 256 samples with a random number added to each sample to represent the noise. It looks as follows.

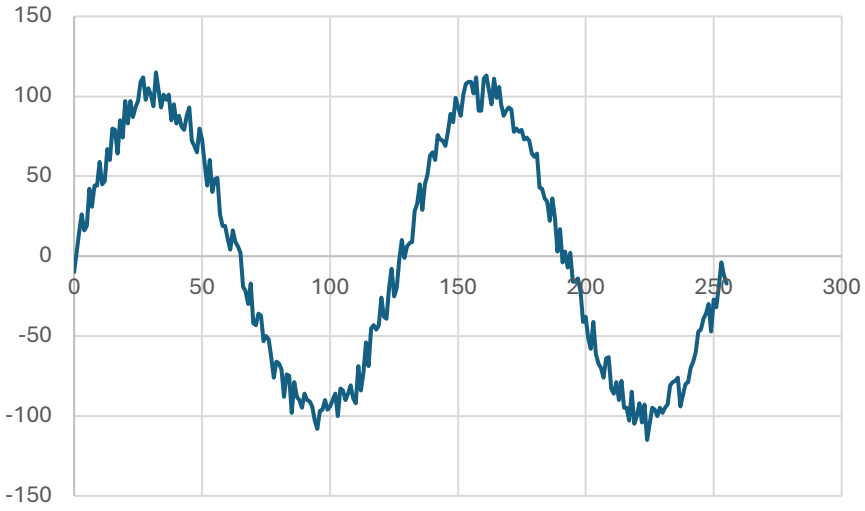

Figure 3. Noisy waveform W[i].

The waveform has been scaled to fit the 8-bit 2's complement integer range so that it's samples can be represented as 256 integer numbers in the range [-128..+127]. . A file, named wave.hex ,with the waveform values represented in hexadecimal notation, ready to be synthesised as a ROM memory in SystemVerilog, is provided on the notes website.

Now, the convolution which will create a smoothed waveform S[i], where i=0..255, is performed on each sample W[i] and it's adjacent samples as follows:

S[i] = W[i-2]*K[0]+W[i-1]*K[1]+W[i]*K[2]*W[i+1]*K[3]+W[i+2]*K[4]

Your task is to design a processor which will read the index i from an input port, then it will execute the above equation using 8-bit 2's complement arithmetic to obtain the value of the smoothed waveform's sample S[i] and finally it will output the 8-bit sample S[i] on the FPGA kit LEDs. In calculating the multiplications, reject the lower 8-bits from the 16-bit double length product. In

additions ignore all overflows. Details of the algorithm you are required to implement are given in the next section.

As a matter of interest, the entire smoothed waveform when the above Gaussian smoothing algorithm is applied to each sample W[i], i=0..255, will look as follows:

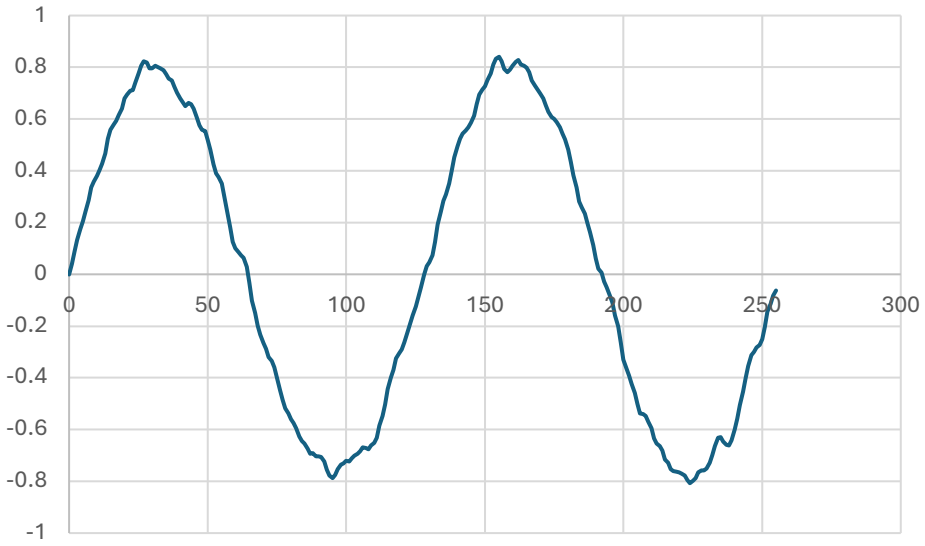

Figure 4. Smoothed waveform S[i] resulting from the convolution.

Implementation of Gaussian smoothing on a picoMIPS

You are required to develop both a smallest possible picoMIPS architecture and a machine-level program for the Gaussian smoothing as explained in the previous section.

The waveform defined in the file wave.hex must be stored in a ROM memory. The five values of the Gaussian kernel, scaled as shown above, can be stored in a ROM or, alternatively, can be represented as immediate 8-bit literals in the program instructions. In the pseudocode below the sample index *i* are read from the switches SW0-SW7 on the FPGA development system and the resulting smoothed sample *S[i]* is displayed on the LEDS LED0-LED7. Switch SW8 provides handshaking functionality as described in the pseudocode. Switch SW9 should act as an active low reset.

During lab demonstrations and testing values of index *i* will be in the range 2..253. The waveform has 256 samples numbered 0..255 but the 5-point convolution cannot be performed on samples 0,1 and 254,255. Once the index *i* is read from the input port, your program should extract the corresponding sample *W[i]* from the ROM memory as well as the adjacent samples *W[i-2], W[i-1], W[i+1]* and *W[i+2]* needed for the convolution and should calculate and display on the LEDs sample *S[i]* of the smoothed waveform. Then the program should loop back to the start and wait for another value of index *i* to be entered.

Pseudocode

1. Wait by polling the switch SW8 until the index *i* is entered on the switches SW0-SW7. Wait while SW8=0. When SW8 becomes 1 (SW8=1) read the index *i* from SW0-SW7.
2. Execute the Gaussian convolution equation for sample *W[i]* of the noisy waveform and display the result *S[i]* on LED0-LED7.
3. Wait until SW8 becomes 0 while displaying the result.
4. Go to step 1.

Input/Output

The input/output functionality can be implemented in several different ways. For example, you can design your own IN and OUT instructions for reading/writing data using external ports. To use fewer hardware resources, you can consider connecting the ALU output, or a register output, directly to the LEDs. You could consider dedicating a specific register number, e.g. register 1 to the input port. In this way, an ADD instruction can be used to read data, e.g. ADD %5, %0, %1 would store the input data in register %5. Be creative and use your imagination!

Design strategy

Develop SystemVerilog code and a separate testbench for each module in your design. Simulate each module in Modelsim. Synthesise each module in Altera Quartus and carefully analyse the synthesis warnings, statistics and RTL diagrams. When you are satisfied that all your picoMIPS modules are correct, write a testbench for the whole design and simulate. Synthesise the whole design and again, carefully analyse the warnings, statistics and RTL diagrams. Test your design either at home or in the laboratory. In Week 10 or 11 (after the Easter Break) you will be asked to demonstrate your design in the Electronics Laboratory. Further information concerning lab demos will be published on the ELEC6234 notes website in due course.

FPGA Development Kit loan

You will be allowed to loan an FPGA development kit from the Electronics Laboratory in Zepler and use it until the lab demo in May. The Laboratory Team will send separate emails with details when the FPGA Kits should be collected and returned.

Formal report

Submit an electronic copy of your report through the electronic handin system by the deadline specified on the ELEC6234 notes website. The report should follow the report template, which will be provided, and the text should not exceed 1500 words in length. SystemVerilog source files must be packaged in a zip file and submitted electronically as a separate file at the same time. In this exercise, 20% of the marks are allocated to the report, its style and organisation, with the remaining 80% for the technical content. As always, bonus marks are awarded for implementation of novel concepts.

Tom Kazmierski, 30 Jan'25